

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

PROJECT REPORT

Hyperlocal Vendor Marketplace System

A Backend System for Localized Inventory & Order Management

Student Name	Rishiraj Singh Rajput
University	VIT Bhopal
Course	Programming in Java (CSE2006)
Project Title	Hyperlocal Vendor Marketplace System

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

This project provides a localized digital commerce backend connecting regional businesses directly with consumers. It aims to digitize inventory and order fulfillment for small-scale vendors, replacing manual, paper-based stock tracking with a lightweight and reliable software system.

The system is implemented entirely in core Java and demonstrates the practical application of object-oriented design, exception handling, multithreading, and file I/O within a single cohesive backend.

3. Problem Statement

Local vendors in developing regional hubs like Sehore lack accessible software to track daily inventory and manage concurrent customer orders. Most such vendors rely on manual registers or memory to track stock, which leads to overselling, lost sales, and disputes with customers.

This project addresses the computing requirements appropriate to this problem by providing a lightweight, robust backend system that can accurately track inventory levels and safely process multiple simultaneous orders without data corruption.

4. Functional Requirements

The system includes three major functional modules:

- **Inventory Management:** Enables CRUD operations for products (Add, Read, Update, Delete), implemented through the `InventoryManager` class.
- **Order Processing:** Validates cart/stock requirements and deducts stock dynamically via the `OrderProcessor` thread class.
- **Receipt Generation:** Utilizes automated input/output structures to generate transactional logs as local text files.

5. Non-functional Requirements

The project prioritizes four distinct system traits:

- **Performance:** Fast in-memory data retrieval using the Java Collections framework (`ArrayList`).
- **Reliability:** Strict error handling strategy utilizing try...catch blocks and a custom checked exception to prevent crashes.
- **Scalability:** Thread-safe operations allowing concurrent user access without corrupting shared inventory state.
- **Maintainability:** Clean, modular OOP file structure separated by responsibility (entity, manager, processor, exception).

6. System Architecture

The application follows a modular client-server conceptual model where the core logic layer (Java application) mediates between user inputs and localized file storage (I/O). Each layer has a single, clearly defined responsibility:

- **Product** — the data/entity layer representing a single vendor item.
- **InventoryManager** — the service layer holding and querying the product catalog.
- **OrderProcessor** — the concurrency layer executing an order as an independent thread.
- **InsufficientStockException** — the domain-specific error-signalling layer.

An expansion module is defined for JDBC database connection, which will eventually replace the in-memory `ArrayList` and local file receipts with persistent relational storage (see Section 14, Future Enhancements).

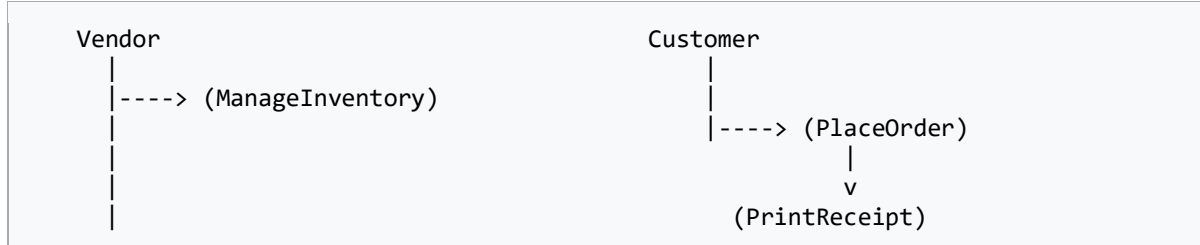
7. Design Diagrams

7.1 Use Case Diagram

Two actors, the Vendor and the Customer, interact with three core use cases exposed by the system:

- **ManageInventory()** — invoked by the Vendor to add or look up products.
- **PlaceOrder()** — invoked by the Customer to purchase a quantity of a product.
- **PrintReceipt()** — triggered automatically after a successful order to generate a receipt.

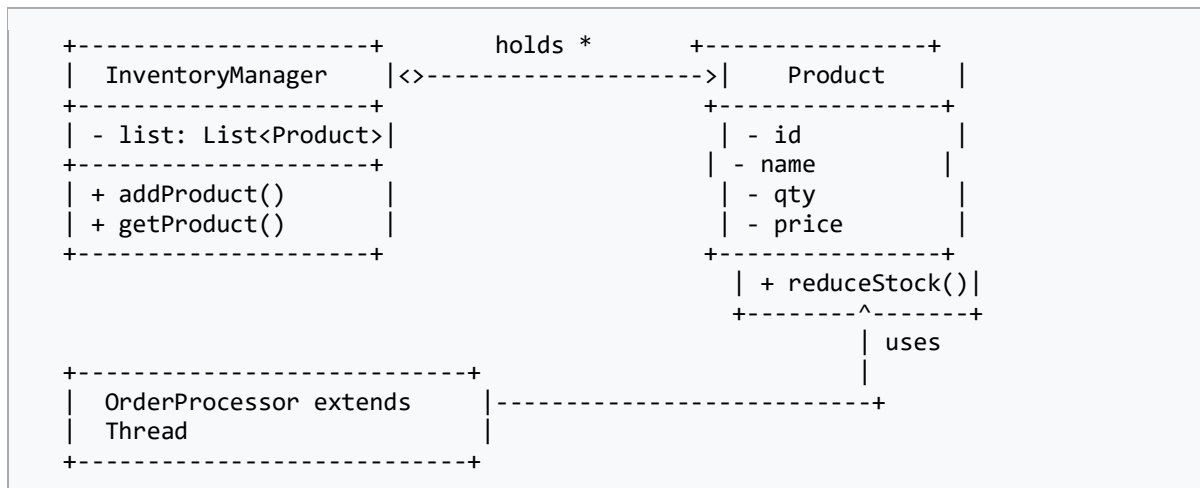
Figure 7.1 — Use Case Overview (text representation)



7.2 Class Diagram

The class diagram outlines the composition and association between `Product`, `InventoryManager`, and the `OrderProcessor` thread class:

Figure 7.2 — Class Relationships (text representation)



```

| - prod: Product          |
| - count: int             |
+-----+
| + run()                  |
| - generateReceipt()      |
+-----+
|           | throws      |
|           | v           |
+-----+
| InsufficientStockException |
| extends Exception         |
+-----+

```

8. Design Decisions & Rationale

- **Dynamic collection:** An ArrayList was chosen over a plain array for the product catalog because it resizes automatically during CRUD operations, avoiding manual capacity management.
- **Synchronization:** Synchronization was applied around stock-reduction to prevent race conditions during high-volume regional sales, ensuring two concurrent orders cannot both read the same stock level before either writes it back.
- **Checked exceptions:** InsufficientStockException is a checked exception, forcing every caller to explicitly acknowledge and handle the out-of-stock scenario rather than letting it fail silently.
- **Case-insensitive lookup:** Product IDs are matched using equalsIgnoreCase() rather than equals(), making the lookup more forgiving of vendor input variance.

9. Implementation Details

The implementation heavily relies on Java Object-Oriented Programming (OOP) for structuring the Product entities. Each product encapsulates its identifier, name, quantity, and price behind private fields with public accessors, and exposes a single controlled mutator, reduceStock(), which enforces the stock-availability invariant before any quantity change is applied.

Multithreading was executed by extending the Thread class in OrderProcessor, adhering to the Java thread lifecycle: each order is created, started via start(), and independently executes its run() method, allowing multiple customers to place orders on the same product concurrently.

Character-oriented streams were utilized via FileWriter to persist receipt data locally. Each receipt is written to a uniquely named text file based on the order timestamp, capturing the item name, quantity, and total amount.

Error handling is layered: reduceStock() throws the checked InsufficientStockException, which propagates up through run() and is caught centrally, where the failure is logged to standard error without crashing the application or other in-flight threads.

10. Screenshots / Results

Terminal execution yields the following logs:

Figure 10.1 — Sample Console Output

```
Added: Locally Sourced Wheat (10kg)
Receipt generated for Locally Sourced Wheat (10kg)
Order failed: Not enough stock for: Locally Sourced Wheat (10kg)
```

The first line confirms successful inventory registration. The second line confirms an order that was within available stock, resulting in a generated receipt file. The third line demonstrates the exception-handling path being correctly triggered when a requested quantity exceeds the remaining stock.

11. Testing Approach

Validation tests were written to trigger custom exceptions, ensuring the `InsufficientStockException` accurately blocks invalid multithreaded transactions. Testing focused on the following scenarios:

- Placing a single order within available stock and confirming the quantity is correctly deducted.
- Placing two concurrent orders whose combined quantity exceeds available stock, confirming only the valid order succeeds and the other is safely rejected.
- Requesting a non-existent product ID and confirming the lookup returns a null result without throwing an unhandled error.
- Confirming a receipt file is generated only for successful orders, and never for rejected ones.

12. Challenges Faced

Managing concurrent access to a single inventory object proved difficult. When two `OrderProcessor` threads attempted to reduce stock on the same `Product` simultaneously, an unsynchronized implementation risked both threads reading the same pre-update quantity and applying overlapping deductions, which silently oversells stock.

The solution involved applying a synchronized block on the target `Product` object during the read-check-update-write sequence, ensuring only one thread could execute that critical section at a time and properly aligning the implementation with Java's built-in monitor-based flow control.

13. Learnings & Key Takeaways

The project successfully demonstrated how to implement high-performing multi-threaded applications and manage Java I/O streams efficiently. Specific takeaways include:

- Practical use of the `synchronized` keyword to protect shared mutable state in a multithreaded environment.
- Designing and using a custom checked exception to model a domain-specific business rule.
- Structuring a small application along clean OOP boundaries so each class has a single, well-defined responsibility.
- Using character streams (`FileWriter`) for straightforward, human-readable local persistence.

14. Future Enhancements

Future iterations will connect the InventoryManager to a persistent MySQL database using the JDBC API to submit queries and parse result sets, replacing the current in-memory ArrayList. Additional planned enhancements include:

- A REST API layer to expose inventory and order operations over HTTP for a web or mobile frontend.
- Migration from FileWriter-based receipts to structured, queryable order records in the database.
- A thread pool (ExecutorService) in place of raw Thread subclassing, for better resource management under high order volume.
- Role-based access separating Vendor-side inventory operations from Customer-side ordering operations.

15. References

- VITyarthi — Build Your Own Project Guidelines.
- Herbert Schildt, "Java: The Complete Reference," 11th Edition, McGraw-Hill Education.